

Git & GitHub — 100 Interview Questions

Sep 28, 2026 ·

100 questions across six sections, moving from fundamentals to real-world scenarios. Each question carries a difficulty tag and a concise model answer an interviewer can check against.

Section	Topic	Questions	Level
A	Git fundamentals and concepts	Q1-Q15	Beginner
B	Core commands and everyday workflow	Q16-Q35	Beginner-Intermediate
C	Branching, merging and rebasing	Q36-Q55	Intermediate
D	Advanced Git internals and tools	Q56-Q70	Advanced
E	GitHub collaboration, PRs, Actions and security	Q71-Q88	Intermediate-Advanced
F	Scenario-based and troubleshooting	Q89-Q100	Advanced

Difficulty legend: **[B]** Beginner · **[I]** Intermediate · **[A]** Advanced.

Section A — Git Fundamentals and Concepts (Q1-Q15)

Q1. What is Git? **[B]**

Git is a free, open-source distributed version control system created by Linus Torvalds in 2005. It tracks changes to files over time, lets many people work on the same codebase, and allows any earlier version to be recovered.

Q2. What is version control, and why is it needed? **[B]**

Version control records every change to a set of files with who made it, when, and why. It enables collaboration, rollback of mistakes, parallel development through branches, and a full audit history.

Q3. What is the difference between centralized and distributed version control? **[B]**

In centralized systems (SVN, CVS) a single server holds the history and clients check out snapshots. In distributed systems (Git, Mercurial) every clone holds the full history, so most operations work offline and there is no single point of failure.

Q4. What is the difference between Git and GitHub? [B]

Git is the version control tool that runs locally. GitHub is a cloud hosting platform for Git repositories that adds collaboration features such as pull requests, issues, Actions, code review and project boards.

Q5. What is a repository? [B]

A repository (repo) is a project folder tracked by Git. Its history and metadata live in the hidden `.git` directory; the files you edit form the working tree.

Q6. Explain the three states / three areas in Git. [B]

Files move through the **working directory** (modified), the **staging area or index** (staged with `git add`), and the **repository** (committed with `git commit`). This lets you choose exactly which changes go into each commit.

Q7. What is a commit? [B]

A commit is a snapshot of the staged project state, identified by a SHA hash. It stores the tree of files, author and committer details, timestamp, message, and pointer(s) to parent commit(s).

Q8. What is a SHA-1 hash in Git? [B]

It is a 40-character hexadecimal checksum computed from an object's content. It uniquely identifies commits, trees, blobs and tags and guarantees integrity: any change in content changes the hash. Newer Git versions also support SHA-256 repositories.

Q9. What is **HEAD** in Git? [B]

HEAD is a pointer to the commit you currently have checked out, usually via the current branch reference. When it points directly at a commit instead of a branch, you are in a “detached HEAD” state.

Q10. What is a branch? [B]

A branch is a lightweight, movable pointer to a commit. It lets you develop features or fixes in isolation; each new commit on the branch moves the pointer forward.

Q11. What is the difference between `git init` and `git clone`? [B]

`git init` creates a new, empty repository in the current folder. `git clone <url>` copies an existing remote repository, including its full history, and sets up `origin` as the remote.

Q12. What is a remote? [B]

A remote is a named reference to another copy of the repository, usually on a server such as GitHub. `origin` is the default name for the clone source; `upstream` commonly refers to the original repo of a fork.

Q13. What is `.gitignore`? [B]

It is a file listing patterns for files Git should not track, such as `node_modules/`, `.env`, build output and logs. It only affects untracked files; already-tracked files must be removed with `git rm --cached`.

Q14. What is the purpose of `git config`? [B]

It sets configuration at system, global (user) or local (repo) level. Typical first-time setup: `git config --global user.name "Name"` and `git config --global user.email "you@example.com"`.

Q15. What are the four types of Git objects? [I]

Blob (file contents), **tree** (directory listing pointing to blobs and subtrees), **commit** (snapshot metadata pointing to a tree and parents), and **annotated tag** (named, signed or annotated pointer to an object).

Section B — Core Commands and Everyday Workflow (Q16–Q35)

Q16. What does `git status` show? [B]

It shows the current branch, staged changes, unstaged modifications, untracked files, and whether the branch is ahead of or behind its upstream.

Q17. What is the difference between `git add .`, `git add -A` and `git add -u`? [B]

In Git 2.x, `git add .` stages new, modified and deleted files in the current directory and below; `git add -A` does the same for the whole repository; `git add -u` stages only modifications and deletions of already-tracked files, not new files.

Q18. How do you write a good commit message? [B]

Use a short imperative subject line of about 50 characters (“Add login validation”), a blank line, then a body explaining what and why. Many teams follow Conventional Commits, e.g. `feat:`, `fix:`, `docs:`.

Q19. What does `git commit --amend` do? [B]

It replaces the last commit with a new one that includes currently staged changes and/or an edited message. Because it rewrites history, avoid amending commits already pushed to a shared branch.

Q20. How do you view commit history? [B]

`git log` shows history. Useful variants: `git log --oneline`, `git log --graph --all --decorate`, `git log -p <file>` (with diffs), and `git log --author="name"`.

Q21. What is the difference between `git diff`, `git diff --staged` and `git diff HEAD`? [I]

`git diff` compares the working directory with the index (unstaged changes). `git diff --staged` compares the index with the last commit (what will be committed). `git diff HEAD` compares the working directory with the last commit (all changes).

Q22. What is the difference between `git fetch` and `git pull`? [B]

`git fetch` downloads new commits and updates remote-tracking branches without touching your working branch. `git pull` runs a fetch followed by a merge (or a rebase with `--rebase`) into the current branch.

Q23. What does `git push` do, and what does `-u` mean? [B]

`git push` uploads local commits to a remote branch. `git push -u origin feature` also sets the upstream tracking relationship, so later `git push` / `git pull` work without arguments.

Q24. What is `git stash`? [B]

It temporarily shelves uncommitted changes so you can switch context with a clean working tree. `git stash pop` reapplies and removes the latest stash; `git stash apply` reapplies but keeps it; `git stash list` shows all stashes.

Q25. How do you stash untracked files as well? [I]

Use `git stash -u` (`--include-untracked`). Use `git stash -a` (`--all`) to include ignored files too.

Q26. How do you remove a file from Git but keep it locally? [B]

Run `git rm --cached <file>`, add it to `.gitignore`, and commit. `git rm <file>` without `--cached` deletes it from disk as well.

Q27. How do you rename or move a file in Git? [B]

Use `git mv old new`. Git does not store renames explicitly; it detects them by content similarity when showing history.

Q28. What is the difference between `git checkout`, `git switch` and `git restore`? [I]

`git checkout` historically did two jobs: switching branches and restoring files. Since Git 2.23 these are split into `git switch <branch>` for branches and `git restore <file>` for files, which is clearer and safer.

Q29. How do you discard local changes to a file? [B]

For unstaged changes: `git restore <file>` (or `git checkout -- <file>`). To unstage a staged file: `git restore --staged <file>`.

Q30. How do you see who changed each line of a file? [I]

`git blame <file>` annotates every line with the commit, author and date of its last change. On GitHub, the Blame view shows the same.

Q31. What does `git show` do? [B]

It displays a single object: for a commit, its metadata and diff. `git show <commit>: <path>` prints a file as it existed at that commit.

Q32. How do you list, add and remove remotes? [B]

`git remote -v` lists remotes with URLs; `git remote add upstream <url>` adds one; `git remote remove <name>` deletes one; `git remote set-url origin <url>` changes a URL.

Q33. What is a tag, and what is the difference between lightweight and annotated tags? [I]

A tag marks a specific commit, typically a release such as `v1.2.0`. A lightweight tag is just a pointer; an annotated tag (`git tag -a v1.0 -m "msg"`) is a full object with tagger, date, message and optional GPG signature. Tags are pushed with `git push origin --tags`.

Q34. What does `git clean` do? [I]

It deletes untracked files from the working tree. Run `git clean -n` first for a dry run, then `git clean -fd` to remove untracked files and directories (`-x` also removes ignored files).

Q35. What are Git aliases? [B]

Shortcuts for longer commands, set with `git config --global alias.st status` or `git config --global alias.lg "log --oneline --graph --all"`.

Section C — Branching, Merging and Rebasing (Q36–Q55)

Q36. How do you create, list, switch and delete branches? [B]

Create: `git branch feature` or `git switch -c feature`. List: `git branch` (local), `git branch -a` (all). Switch: `git switch feature`. Delete: `git branch -d feature` (safe) or `-D` (force). Delete remote: `git push origin --delete feature`.

Q37. What is `git merge`? [B]

It integrates the history of another branch into the current branch. Git finds the common ancestor and combines changes, creating a merge commit when both branches have diverged.

Q38. What is a fast-forward merge? [I]

When the current branch has no new commits since the other branch diverged, Git simply moves the pointer forward; no merge commit is created. `git merge --no-ff` forces a merge commit to preserve the feature's grouping in history.

Q39. What is a three-way merge? [I]

When both branches have new commits, Git compares the two branch tips against their common ancestor (merge base) and creates a merge commit with two parents.

Q40. What is a merge conflict, and how do you resolve it? [B]

A conflict occurs when both branches change the same lines, or one edits a file the other deletes. Git marks conflicts with `<<<<<<`, `=====` and `>>>>>>`. Edit the file to the correct result, `git add` it, then `git commit` (or `git merge --continue`). `git merge --abort` cancels.

Q41. What is `git rebase`? [I]

It moves or replays your branch's commits on top of another base commit, producing a linear history. Example: on `feature`, `git rebase main` re-applies feature commits after the latest `main`.

Q42. Merge vs rebase: what is the difference and when should each be used? [I]

Merge preserves true history and is non-destructive, but adds merge commits. Rebase gives a clean linear history but rewrites commit hashes. Rebase local or private branches to tidy them; use merge for integrating shared branches.

Q43. What is the golden rule of rebasing? [I]

Never rebase commits that have been pushed and that others may have based work on. Rewriting shared history forces collaborators to reconcile diverged branches.

Q44. What is interactive rebase? [I]

`git rebase -i HEAD~n` opens an editor listing the last n commits, where you can `pick`, `reword`, `edit`, `squash`, `fixup`, `drop` or reorder them. It is used to clean up history before opening a pull request.

Q45. What is squashing commits? [I]

Combining several commits into one, via interactive rebase, `git merge --squash`, or GitHub's "Squash and merge" option. It keeps the main branch history concise.

Q46. What is `git cherry-pick`? [I]

It applies the changes from specific commit(s) onto the current branch as new commits: `git cherry-pick <sha>`. Common for back-porting a hotfix to a release branch.

Q47. What is a detached HEAD, and how do you recover from it? [I]

It happens when you check out a commit, tag or remote branch directly. New commits there belong to no branch. To keep them, run `git switch -c new-branch` before leaving; otherwise find them later with `git reflog`.

Q48. What is a remote-tracking branch? [I]

A local read-only reference such as `origin/main` that records where the remote branch was at the last fetch. You compare and merge from it but do not commit to it directly.

Q49. What does it mean for a branch to be "ahead" or "behind"? [B]

"Ahead by 2" means your local branch has two commits not on its upstream; "behind by 3" means the upstream has three commits you have not integrated. `git status` and `git branch -vv` report this.

Q50. Describe the Git Flow branching model. [I]

It uses long-lived `main` (production) and `develop` (integration) branches, plus short-lived `feature/*`, `release/*` and `hotfix/*` branches. It suits scheduled, versioned releases but adds overhead.

Q51. What is GitHub Flow? [I]

A lightweight model: `main` is always deployable; create a feature branch, commit, open a pull request, review, run CI, merge to `main`, and deploy. It suits continuous delivery.

Q52. What is trunk-based development? [A]

Developers integrate small changes into the trunk (`main`) at least daily, using very short-lived branches and feature flags to hide incomplete work. It reduces merge pain and supports CI/CD.

Q53. What merge strategies does Git support? [A]

The default for two branches is `ort` (which replaced `recursive` in Git 2.34). Others include `octopus` (many branches), `ours` (keep current tree entirely) and `subtree`. Options such as `-X ours` / `-X theirs` favor one side only in conflicting hunks.

Q54. How do you find the common ancestor of two branches? [A]

`git merge-base main feature` prints the best common ancestor commit, the base Git uses for a three-way merge or rebase.

Q55. What is `git rerere`? [A]

“Reuse recorded resolution.” When enabled (`git config rerere.enabled true`), Git remembers how you resolved a conflict and reapplies the same resolution automatically if it recurs, useful during repeated rebases.

Section D — Advanced Git Internals and Tools (Q56–Q70)**Q56. Explain `git reset` and its `--soft`, `--mixed` and `--hard` modes. [I]**

`git reset <commit>` moves the current branch pointer. `--soft` keeps changes staged; `--mixed` (default) keeps changes in the working tree but unstaged; `--hard` discards all changes in index and working tree. `--hard` can lose uncommitted work.

Q57. What is the difference between `git reset` and `git revert`? [I]

`git reset` rewrites history by moving the branch pointer backwards, suitable for local, unpushed commits. `git revert <sha>` creates a new commit that undoes a previous commit, safe for shared branches because history is preserved.

Q58. What is `git reflog`? [A]

A local log of every position `HEAD` and branch tips have pointed to, including after resets, rebases and deleted branches. It is the primary tool for recovering “lost” commits: `git reset --hard HEAD@{2}` or `git branch recover <sha>`. Entries expire (90 days by default).

Q59. What is `git bisect`? [A]

A binary search through history to find the commit that introduced a bug. Run `git bisect start`, mark `git bisect bad` and `git bisect good <sha>`, test each suggested commit, and finish with `git bisect reset`. `git bisect run <script>` automates it.

Q60. What are Git hooks? [A]

Scripts in `.git/hooks` that run on events: client-side (`pre-commit`, `commit-msg`, `pre-push`) and server-side (`pre-receive`, `update`, `post-receive`). They enforce linting, tests or message formats. Tools like Husky or pre-commit share hooks across a team.

Q61. What are Git submodules? [A]

A way to embed another repository at a fixed commit inside your repo: `git submodule add <url> path`. Clone with `git clone --recurse-submodules` or run `git submodule update --init --recursive`. They pin exact versions but add workflow complexity.

Q62. Submodule vs subtree: what is the difference? [A]

A submodule stores only a pointer to an external repo's commit, so contributors must initialize it separately. A subtree copies the external project's files and history into your repo, making cloning simple but updates heavier.

Q63. What is Git LFS? [A]

Git Large File Storage replaces large binaries (videos, datasets, models) with small pointer files, storing the real content on a separate server. Configure with `git lfs install` and `git lfs track "*.psd"`.

Q64. What is a bare repository? [A]

A repository with no working tree, only the Git data (`git init --bare`). Servers use bare repos as push targets because no one edits files there directly.

Q65. How does Git store data internally: snapshots or deltas? [A]

Conceptually Git stores snapshots: each commit points to a full tree, with unchanged files reusing the same blobs. For storage efficiency, `git gc` compresses objects into packfiles using delta compression.

Q66. What does `git gc` do? [A]

Garbage collection packs loose objects into packfiles, removes unreachable objects past their expiry, and optimizes the repository. Git runs it automatically at times; `git gc --aggressive` does deeper optimization.

Q67. What is `git worktree`? [A]

It lets one repository have multiple working directories checked out to different branches at once: `git worktree add ../hotfix hotfix-branch`. Useful for working on a hotfix without stashing feature work.

Q68. What are shallow clones and sparse checkout? [A]

A shallow clone (`git clone --depth 1`) fetches only recent history, speeding up CI. Sparse checkout (`git sparse-checkout set dir1 dir2`) limits the working tree to selected folders, useful in large monorepos.

Q69. How do you sign commits, and why? [A]

Configure a GPG, SSH or S/MIME key and use `git commit -S` (or `git config commit.gpgsign true`). Signing proves authorship; GitHub shows a “Verified” badge and branch rules can require signed commits.

Q70. What is `git filter-repo` used for? [A]

It rewrites whole-repository history, for example to purge a leaked secret or large file from every commit, or to extract a subdirectory. It replaces the older, slower `git filter-branch`; BFG Repo-Cleaner is an alternative. All collaborators must re-clone afterwards.

Section E — GitHub Collaboration, PRs, Actions and Security (Q71–Q88)**Q71. What is a fork, and how is it different from a clone? [B]**

A fork is a server-side copy of someone else’s repository under your GitHub account, used to propose changes without write access. A clone is a local copy on your machine. The usual flow is fork, clone your fork, push to it, then open a pull request.

Q72. What is a pull request (PR)? [B]

A request to merge changes from one branch (or fork) into another. It provides a diff view, discussion, line comments, reviews, CI status checks and a merge button, making it the core of code review on GitHub.

Q73. What are the three merge options on a GitHub PR? [I]

Create a merge commit keeps all commits plus a merge commit; **Squash and merge** combines the PR into one commit; **Rebase and merge** replays commits linearly onto the base without a merge commit. Admins can enable or disable each.

Q74. How do you keep a fork in sync with the original repository? [I]

Add the original as `upstream`, then `git fetch upstream`, `git switch main`, `git merge upstream/main` (or `rebase`), and `git push origin main`. GitHub's "Sync fork" button does this in the UI.

Q75. What is a draft pull request? [B]

A PR marked as work in progress. It cannot be merged and does not request reviews from code owners until marked "Ready for review", which is useful for early feedback and CI runs.

Q76. What are GitHub Issues? [B]

Trackers for bugs, tasks and feature requests, with labels, assignees, milestones and templates. Writing `Fixes #42` or `Closes #42` in a PR description closes the issue automatically when the PR merges into the default branch.

Q77. What are branch protection rules and rulesets? [I]

Settings that guard branches like `main`: require PR reviews, passing status checks, signed commits, linear history or up-to-date branches, and block force-pushes and deletions. Rulesets are the newer, layered way to apply such rules across branches, tags and repositories.

Q78. What is a CODEOWNERS file? [I]

A file (in `.github/`, root or `docs/`) mapping paths to responsible users or teams, e.g. `/api/ @org/backend-team`. Owners are auto-requested for review, and branch rules can require their approval.

Q79. What is GitHub Actions? [I]

GitHub's built-in CI/CD and automation platform. Workflows are YAML files in `.github/workflows/`, triggered by events such as `push`, `pull_request`, `schedule` or `workflow_dispatch`, and run jobs on GitHub-hosted or self-hosted runners.

Q80. Explain workflows, jobs, steps, actions and runners. [I]

A **workflow** is an automated process defined in one YAML file. It contains **jobs** that run on a **runner** (a VM or container), in parallel by default or ordered with `needs`. Each job has **steps**, which run shell commands or reusable **actions** such as `actions/checkout`.

Q81. How are secrets handled in GitHub Actions? [I]

Store them in repository, environment or organization secrets and reference them as `${{ secrets.NAME }}`; they are masked in logs. Prefer OIDC federation for cloud credentials over long-lived keys, and apply least-privilege `permissions:` to `GITHUB_TOKEN`.

Q82. What is a matrix build? [A]

`strategy.matrix` runs one job across combinations of variables, e.g. several OS and language versions, in parallel, to test compatibility with a single definition.

Q83. What is the difference between SSH and HTTPS authentication with GitHub? [B]

HTTPS uses a personal access token (passwords are no longer accepted) or a credential manager. SSH uses a key pair: add the public key to your GitHub account and use `git@github.com:user/repo.git` URLs. Both are secure; SSH avoids repeated token entry.

Q84. What are fine-grained personal access tokens? [I]

Tokens scoped to specific repositories and specific permissions with mandatory expiry, replacing broad classic tokens. They follow the principle of least privilege.

Q85. What GitHub features help with security? [I]

Dependabot alerts and version updates, secret scanning with push protection, code scanning (CodeQL), security advisories, the dependency graph, and required 2FA for contributors.

Q86. What are GitHub Pages and GitHub Releases? [B]

GitHub Pages hosts static websites directly from a repository branch or an Actions build. Releases package a tag with release notes and downloadable binaries or assets for users.

Q87. What are GitHub Projects, Discussions and Wikis? [B]

Projects are table, board and roadmap views for planning issues and PRs. Discussions provide a community forum for Q&A and ideas. Wikis host long-form documentation alongside the repo.

Q88. What is the GitHub CLI? [I]

`gh` is the official command-line tool for GitHub: `gh repo clone`, `gh pr create`, `gh pr checkout 123`, `gh issue list`, `gh run watch`. It brings PR and issue workflows to the terminal and scripts.

Section F — Scenario-Based and Troubleshooting (Q89–Q100)

Q89. You committed to the wrong branch (not yet pushed). How do you fix it? [I]

Note the commit SHA, switch to the correct branch and run `git cherry-pick <sha>`. Return to the wrong branch and remove it with `git reset --hard HEAD~1`. If the target branch doesn't exist yet, `git branch correct-branch` then reset the current one.

Q90. You accidentally committed a password or API key and pushed it. What do you do? [A]

First, revoke and rotate the credential immediately; assume it is compromised. Then purge it from history with `git filter-repo` or BFG, force-push, ask collaborators to re-clone, and contact GitHub Support to clear cached views if needed. Prevent recurrence with `.gitignore`, environment variables and secret-scanning push protection.

Q91. How do you undo a commit that is already pushed to a shared branch? [I]

Use `git revert <sha>` and push the new commit. It undoes the change without rewriting history, so teammates are unaffected. For a merge commit, use `git revert -m 1 <merge-sha>`.

Q92. Your push is rejected with “non-fast-forward”. Why, and how do you resolve it? [B]

The remote has commits you don't have. Run `git pull --rebase` (or `git pull`), resolve any conflicts, then push again. Do not force-push over others' work.

Q93. When is a force push acceptable, and what is `--force-with-lease`? [I]

Only on your own feature branch after rebasing or amending, never on shared branches like `main`. `git push --force-with-lease` refuses to overwrite the remote if someone else pushed since your last fetch, making it far safer than `--force`.

Q94. You ran `git reset --hard` and lost commits. How do you recover them? [A]

Run `git reflog`, find the entry before the reset (e.g. `HEAD@{1}`), and run `git reset --hard HEAD@{1}` or create a branch at that SHA. Uncommitted changes discarded by `--hard` generally cannot be recovered.

Q95. You deleted a branch by mistake. How do you restore it? [I]

Find the branch tip's SHA in `git reflog` (or the deletion output), then `git branch <name> <sha>`. On GitHub, a recently deleted PR branch can be restored from the PR page.

Q96. How do you combine your last 5 commits into one before opening a PR? [I]

`git rebase -i HEAD~5`, keep the first as `pick`, mark the rest `squash` or `fixup`, edit the message, then `git push --force-with-lease`. Alternatively `git reset --soft HEAD~5` followed by one new commit.

Q97. You have a large, long-running feature branch that keeps conflicting with `main`. How would you manage it? [A]

Integrate `main` frequently (daily rebase or merge) to keep conflicts small, split the work into smaller incremental PRs, hide unfinished work behind feature flags, and enable `rerere`. Communicate with owners of overlapping code.

Q98. A file listed in `.gitignore` is still being tracked. Why, and how do you fix it? [B]

`.gitignore` only affects untracked files. Stop tracking it with `git rm --cached <file>` (or `-r` for folders), commit, and it will be ignored from then on.

Q99. A bug appeared somewhere in the last 200 commits. How do you find the cause efficiently? [A]

Use `git bisect`: mark the current commit bad and a known-good commit good. Git binary-searches, needing about 8 tests for 200 commits. Automate with `git bisect run npm test` if a test reproduces the bug.

Q100. What Git and GitHub best practices would you recommend for a team? [I]

Small, focused commits with clear messages; short-lived branches with a consistent naming scheme; all changes via reviewed PRs with CI checks; protected `main` with required reviews; no secrets in repositories; signed commits where needed; tagged, documented releases; and a written contribution guide (`CONTRIBUTING.md`).